Design card — Wacky Wheels Control Tower · team penberthy-pushers

Architecture. A **Coordinator** plus three specialists, all calling the enterprise tools (`POST /act`) through one validated, audited `ToolSession`.

Flow: `GET /events` → Coordinator: LLM extraction → grounding in the knowledge graph → decide *noise / escalate / act* → specialists traverse the graph and plan → validated `POST /act` calls → `action_log.jsonl` + per-event trace (summary, reason, graph path, audit trail) → control-tower UI.

Agent	Responsibility	Tools
Coordinator	LLM understanding, graph grounding, noise / escalate / act decision, routing, guardrails, summary	`escalate`
SupplyChainPlanner	Delays and outages: finds affected builds by graph traversal, moves each to the earliest feasible slot (same plant, else transfer) with live capacity	`mes.get_schedule`, `mes.reschedule`, `mes.transfer`
CustomerDealerAgent	One `dealer.notify` per affected dealer (only its orders); loaners by tier, promised date and pool limit	`dealer.notify`, `crm.loaner_availability`, `crm.assign_loaner`
QualityAgent	Lot → vehicles: hold lot, stop sale of unsold vehicles, contact owners of sold ones	`quality.hold_lot`, `dealer.stop_sale`, `crm.notify_customer`

Knowledge graph. 12 classes and 19 typed relations; 17,040 nodes and 28,251 edges, built from the CSVs in under 0.2 s. Every traversal returns the records *and* the typed path it walked (for example `Shipment -CARRIES-> Part <-HAS_PART- Model <-OF_MODEL- ProductionOrder <-FULFILLED_BY- CustomerOrder -> Dealer / Customer`); each trace cites one path per incident. Diagram `ontology.svg`; export `ontology.json`, `ontology.mmd`; and the Ontology tab.

LLM versus code. The LLM only does what needs language: classify, split multi-incident messages, and copy entity mentions and date phrases *verbatim* (JSON-schema output, temperature 0, one call per event). Code does everything that must be exact: alias and graph name resolution, relative dates from 2027-03-08 ("next Friday"), shipment and lot identification, handbook rules and slot planning. The model never invents an id or a date, and the same event always yields the same actions. A rule-based extractor implements the same schema as offline fallback and cross-checks every event.

Guardrails. (1) Grounding gate: anything that cannot be resolved uniquely in the graph becomes *unconfirmed* and is escalated. (2) Rumors: escalate only. (3) Noise: zero tool calls. (4) Every call validated against `/tools` before sending; capacity read live before each move. (5) Transport errors retried with backoff; business errors never blindly retried, the planner refreshes and tries the next slot; no feasible slot → escalate. (6) LLM failure → circuit breaker → rules. (7) Compound events share one capacity view and one loaner allocation; each dealer gets exactly one message.

Evaluation. `verify.py` replays `action_log.jsonl` on a fresh copy of the organizers' world and compares it with a *separate* handbook implementation that shares no code with the agents. **Run 1: 15/15 events pass, 879 calls, 0 failed, 766 writes;** all 50 moved orders (45 reschedules, 5 transfers) match an independent earliest-slot plan. The six vague messages from the organizers' explainer: 5 escalations, 1 noise, 0 writes. A fresh rerun reproduces all 877 operational calls exactly, in the same order (only the wording of the 2 escalation reasons varies), and the offline rules-only fallback produces the same actions.

Model used (citation). Google Gemini 3.5 Flash-Lite (`gemini-3.5-flash-lite`, Gemini API v1beta, free tier) for event understanding only: 15 calls, 6,761 input + 2,187 output tokens, 0 retries. No other ML model; planning is deterministic Python. Optional, unused in Run 1: Anthropic Claude via the `anthropic` SDK.

Interpretation choices. "Next Wednesday" means Wednesday of next week, and likewise for every weekday; this is consistent with the shipment and lot tables. Transfers go to the earliest free day at any plant building the model (`HTW_TRANSFER=after_restart` is the switchable alternative; it changes 5 orders).

What we would improve. Live comparison of small models (Claude Haiku vs Gemini Flash-Lite) on extraction accuracy, cost and latency; an adversarial event set (typos, conflicting dates, unknown suppliers); human approval in the UI before large write batches; an optimizer instead of greedy slots when disruptions compete for capacity.